

Array

In Python, Array is an object that provide a mechanism for storing several data items with only one identifier, thereby simplifying the task of data management. Array is beneficial if you need to store group of elements of same datatype.

In Python, Arrays can increase or decrease their size dynamically.



Group of Students



Group of Employees



Group of Dogs

Group of Integer- 10, 2, 40, 5, 6

Group of Float – 15.4, 25.4, 6.5

- Arrays can store only one type of data.
- In Python, The size of array is not fixed. Array can increase or decrease their size dynamically.
- Array and List are not same.
- Array uses less memory than List.

Why we need Array ?

101, 102, 103, 104, 105

stu1_roll = 101

stu2_roll = 102

stu3_roll = 103

stu4_roll = 104

stu5_roll = 105

print(stu1_roll)

print(stu2_roll)

print(stu3_roll)

print(stu4_roll)

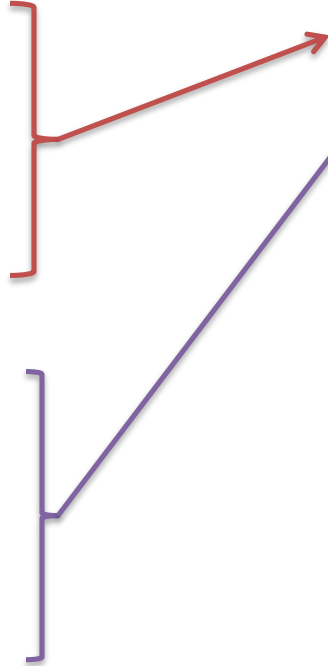
print(stu5_roll)

101, 102, 103, 104, 105

stu_roll = array('i', [101,102,103,104,105])

for element in stu_roll:

print(element)



Type of Array

- One Dimensional Array / One D Array – Single Row Multiple Columns

Ex:- Student's Roll Number

[101, 102, 103, 104, 105]

- Multi-Dimensional Array / Multi D Array – Multiple Rows Multiple Columns

Ex:- Student's Subject Marks

[40, 60, 70, 80, 30],

1st students Marks

[50, 40, 60, 30, 40]

2nd students Marks

Note - Python does not support Multi-Dimensional Array but we can create Multi Dimensional Array using third party packages like numpy.

One Dimensional Array

Single Row Multiple Columns

Ex:- [101, 102, 103, 104, 105]

Import Array Module

Two way to import array module:-

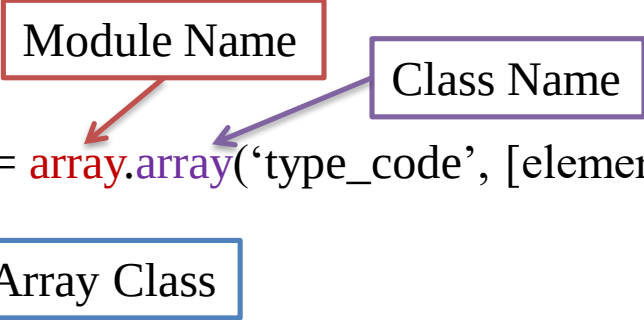
- `import array` – This will import the entire array module.
- `from array import *` - This will import all class, objects, variable etc from array module. Here * means All.

Creating and Initializing One-D Array

Syntax:-

```
import array
```

Module Name



Class Name

```
array_name = array.array('type_code', [elements])
```

Object of Array Class

Ex:-

```
import array
```

```
stu_roll = array.array('i', [101, 102, 103, 104, 105])
```

Creating and Initializing One-D Array

Syntax:-

```
from array import *  
array_name = array('type code', [elements])
```

Class Name



Object of Array Class



Ex:-

```
from array import *  
stu_roll = array('i', [101, 102, 103, 104, 105])
```


Creating Empty One-D Array

Syntax:-

```
from array import *  
array_name = array('type code', [ ])
```

Class Name



Object of Array Class



Ex:-

```
from array import *  
stu_roll = array('i', [ ])
```

Type Code

Type Code	C Type	Python Type	Size in bytes
<i>b</i>	Signed char	int	1
<i>B</i>	Unsigned char	int	1
<i>h</i>	Signed short	int	2
<i>H</i>	Unsigned short	int	2
<i>i</i>	Signed int	int	2
<i>I</i>	Unsigned int	int	2
<i>l</i>	Signed long	int	4
<i>L</i>	Unsigned long	int	4
<i>q</i>	Signed long long	int	8
<i>Q</i>	Unsigned long long	int	8
<i>f</i>	Float	float	4
<i>d</i>	Double	float	8

index

An index represents the position number of an array's element.

```
from array import *
```

```
stu_roll = array('i', [101, 102, 103, 104, 105])
```

Python interpreter allocates 5 blocks of memory and stores the elements

Index always starts with 0



101	102	103	104	105
0	1	2	3	4

101	102	103	104	105
<code>stu_roll[0]</code>	<code>stu_roll[1]</code>	<code>stu_roll[2]</code>	<code>stu_roll[3]</code>	<code>stu_roll[4]</code>

Accessing One-D Array Elements

```
from array import *
```

```
stu_roll = array('i', [101, 102, 103, 104, 105])
```

```
print(stu_roll[0])
```

```
print(stu_roll[1])
```

```
print(stu_roll[2])
```

```
print(stu_roll[3])
```

```
print(stu_roll[4])
```

101	102	103	104	105
stu_roll[0]	stu_roll[1]	stu_roll[2]	stu_roll[3]	stu_roll[4]

Accessing Array using for loop

```
from array import *  
stu_roll = array('i', [101, 102, 103, 104, 105])
```

Without index

```
for element in stu_roll:  
    print(element)
```

With index

```
n = len(stu_roll)  
for i in range(n):  
    print(stu_roll[i])
```

Accessing Array using while loop

```
from array import *  
stu_roll = array('i', [101, 102, 103, 104, 105])  
  
n = len(stu_roll)  
i = 0  
while i < n :  
    print(stu_roll[i])  
    i+=1
```

append ()

This method is used to add an element at the end of the existing array.

Syntax:-

```
array_name.append(new_element)
```



Object of Array Class

Getting User input

```
from array import *  
stu_roll = array('i', [ ])  
n = int(input("How many elements? "))  
for i in range(n):  
    stu_roll.append(int(input("Enter Element: ")))  
for i in range(len(stu_roll)):  
    print(stu_roll[i])
```


insert()

This method is used to insert an element in a particular position of the existing array.

Syntax:-

```
array_name.insert(position_number, new_element)
```

pop()

This method is used to remove last element from the existing array.

Syntax:-

```
array_name.pop( )
```

pop (n)

This method is used to remove an element specified by position number, from the existing array and returns removed element.

Syntax:-

```
array_name.pop(position_number)
```

remove()

This method is used to remove first occurrence of given element from the existing array. If it doesn't found the element, shows `valueError`.

Syntax:-

```
array_name.remove(element)
```

index()

This method returns position number of first occurrence of given element in the array. If it doesn't found the element, shows `valueError`.

Syntax:-

```
array_name.index(element)
```

reverse ()

This method is used to reverse the order of elements in the array.

Syntax:-

```
array_name.reverse( )
```

extend()

This method is used to append another array or iterable object at the end of the array

Syntax:-

```
array_name.extend(arr)
```

Slicing on Arrays

Slicing on arrays can be used to retrieve a piece of the array that contains a group of elements. Slicing is useful to retrieve a range of elements.

Syntax:-

```
new_array_name = array_name[start:stop:stepsize]
```